UNIVERSITY ECHAHID HAMA LAKHDAR — EL OUED
Department of Computer Science | Master 2 — Artificial Intelligence and Data Science

LAB 3 — FAST R-CNN & FASTER R-CNN

Practical Python code and checkpoint questions | Google Colab

Task 0 — Google Colab setup

Prepare a Colab runtime (GPU recommended) and install the Selective Search extension.

Cell 0.1 | Install the additional OpenCV module

```
%pip -q install --upgrade --no-deps opencv-contrib-python-headless
```

Cell 0.2 | Import libraries and check the runtime

```
import time
import cv2
import numpy as np
import torch
import matplotlib.pyplot as plt
from PIL import Image
from matplotlib.patches import Rectangle
from torchvision.transforms import functional as TF

assert hasattr(cv2, 'ximgproc') and hasattr(
    cv2.ximgproc, 'segmentation'
), 'Restart the Colab runtime, then run the cells again.'

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('PyTorch:', torch.__version__)
print('OpenCV:', cv2.__version__)
print('Device:', device)
```

Checkpoint questions

1. Why is a GPU useful for deep feature extraction?

2. What is the difference between installing an OpenCV package and importing cv2?

Task 1 — Image and region proposals

Load a reproducible example and obtain external candidate regions with Selective Search.

Cell 1.1 | Download and display the test image

```
from io import BytesIO
import requests

url = 'https://images.cocodataset.org/val2017/000000039769.jpg'
try:
    response = requests.get(url, timeout=25)
    response.raise_for_status()
    image_raw = np.array(
        Image.open(BytesIO(response.content)).convert('RGB')
    )
except Exception:
    from skimage import data
    image_raw = data.chelsea()  # Offline fallback: cat image

image_rgb = cv2.resize(image_raw, (512, 384))
H, W = image_rgb.shape[:2]
plt.figure(figsize=(9, 5))
plt.imshow(image_rgb)
plt.axis('off')
plt.title('Test image')
plt.show()
```

Cell 1.2 | Generate Selective Search proposals

```
ss = cv2.ximgproc.segmentation.createSelectiveSearchSegmentation()
ss.setBaseImage(cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR))
ss.switchToSelectiveSearchFast()
raw_rectangles = ss.process()

proposals = []
seen = set()
for x, y, w, h in raw_rectangles:
    if w < 24 or h < 24 or w * h > 0.95 * W * H:
        continue
    box = (int(x), int(y), int(x + w), int(y + h))
    if box not in seen:
        proposals.append(box)
        seen.add(box)
    if len(proposals) >= 350:
        break

proposals = np.asarray(proposals, dtype=np.float32)
assert len(proposals) > 0, 'No valid proposals were generated.'
print('Candidate regions:', len(proposals))
```

Cell 1.3 | Draw the first candidate regions

```
fig, ax = plt.subplots(figsize=(9, 6))
ax.imshow(image_rgb)
for x1, y1, x2, y2 in proposals[:35]:
    ax.add_patch(Rectangle(
        (x1, y1), x2 - x1, y2 - y1,
        fill=False, edgecolor='#2563eb', linewidth=0.8
    ))
ax.set_title('External proposals: Selective Search')
ax.axis('off')
plt.show()
```

Checkpoint questions

1. Why are candidate regions not yet object detections?
2. What happens if Selective Search misses an actual object?
3. What limitation of the original R-CNN pipeline remains in Fast R-CNN?

Task 2 — Fast R-CNN: shared features and RoI Pooling

Apply the backbone once to the whole image and pool each candidate region.

Cell 2.1 | Load a pretrained feature extractor

```
from torch import nn
from torchvision.models import resnet18, ResNet18_Weights

backbone_weights = ResNet18_Weights.DEFAULT
network = resnet18(weights=backbone_weights).to(device).eval()
feature_extractor = nn.Sequential(
    *list(network.children())[:-2]
).to(device).eval()

mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]

def prepare_tensor(rgb):
    tensor = TF.to_tensor(rgb).to(device)
    return TF.normalize(tensor, mean, std).unsqueeze(0)

whole_image = prepare_tensor(image_rgb)
with torch.inference_mode():
    feature_map = feature_extractor(whole_image)
print('Image tensor:', tuple(whole_image.shape))
print('Feature map:', tuple(feature_map.shape))
```

Cell 2.2 | Pool multiple regions from one feature map

```
from torchvision.ops import roi_pool

selected_boxes = torch.as_tensor(
    proposals[:24], dtype=torch.float32, device=device
)
spatial_scale = feature_map.shape[-1] / W
with torch.inference_mode():
    pooled_features = roi_pool(
        feature_map, [selected_boxes],
        output_size=(7, 7), spatial_scale=spatial_scale
    )
print('RoI Pooling output:', tuple(pooled_features.shape))
print('One region:', tuple(pooled_features[0].shape))
```

Cell 2.3 | View backbone feature channels

```
fig, axes = plt.subplots(1, 4, figsize=(12, 3))
for ax, channel in zip(axes, range(4)):
    ax.imshow(feature_map[0, channel].detach().cpu(),
              cmap='viridis')
    ax.set_title(f'Channel {channel}')
    ax.axis('off')
plt.tight_layout()
plt.show()
```

Checkpoint questions

1. Why does RoI Pooling produce fixed-size outputs for differently sized boxes?
2. How does shared CNN computation differ from Lab 2 (original R-CNN)?
3. What does spatial_scale represent?

Task 3 — Feature-extraction cost and Fast R-CNN heads

Compare separate region passes with shared features; inspect classification and box-regression outputs.

Cell 3.1 | Compare independent and shared CNN passes

```
def synchronize():
    if device.type == 'cuda':
        torch.cuda.synchronize()

sample_boxes = proposals[:12].astype(int)
with torch.inference_mode():
    synchronize()
    start = time.perf_counter()
    for x1, y1, x2, y2 in sample_boxes:
        crop = image_rgb[y1:y2, x1:x2]
        crop = cv2.resize(crop, (224, 224))
        _ = feature_extractor(prepare_tensor(crop))
    synchronize()
    separate_time = time.perf_counter() - start

    synchronize()
    start = time.perf_counter()
    shared_map = feature_extractor(whole_image)
    _ = roi_pool(
        shared_map, [torch.as_tensor(
            sample_boxes, dtype=torch.float32, device=device
        )], output_size=(7, 7), spatial_scale=spatial_scale
    )
    synchronize()
    shared_time = time.perf_counter() - start

print(f'Separate passes: {separate_time:.3f} s')
print(f'Shared pass + RoI Pooling: {shared_time:.3f} s')
```

Cell 3.2 | Inspect a Fast R-CNN-style prediction head

```
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

head = FastRCNNPredictor(512, num_classes=3).to(device).eval()
with torch.inference_mode():
    region_vectors = pooled_features.mean(dim=(2, 3))
    class_logits, box_offsets = head(region_vectors)
print('Class logits:', tuple(class_logits.shape))
print('Box offsets:', tuple(box_offsets.shape))
```

Technical note. The small predictor above has RANDOM weights. Its output shapes illustrate the classification and box-regression branches; its numeric predictions are not meaningful detections.

Checkpoint questions

1. Which operation is repeated in original R-CNN but shared in Fast R-CNN?
2. Why are there both classification scores and bounding-box offsets?
3. Why should the untrained predictor outputs not be interpreted as accuracy?

Task 4 — Real Fast R-CNN-style detection using external proposals

Reuse trained RoI heads with Selective Search proposals; inspect actual object predictions.

Cell 4.1 | Load a pretrained detector and its RoI heads

```
from torchvision.models.detection import (
    fasterrcnn_resnet50_fpn,
    FasterRCNN_ResNet50_FPN_Weights
)

weights = FasterRCNN_ResNet50_FPN_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn(
    weights=weights,
    min_size=384,
    max_size=512,
    box_score_thresh=0.35,
    rpn_pre_nms_top_n_test=600,
    rpn_post_nms_top_n_test=150
).to(device).eval()
categories = weights.meta['categories']
image_tensor = TF.to_tensor(image_rgb).to(device)
print('Loaded pretrained detector with', len(categories),
      'category entries')
```

Cell 4.2 | Run the trained RoI heads on external proposals

```
with torch.inference_mode():
    transformed, _ = model.transform([image_tensor], None)
    pyramid = model.backbone(transformed.tensors)
    new_h, new_w = transformed.image_sizes[0]
    sx, sy = new_w / W, new_h / H

    external_boxes = torch.as_tensor(
        proposals[:300], dtype=torch.float32, device=device
    ).clone()
    external_boxes[:, [0, 2]] *= sx
    external_boxes[:, [1, 3]] *= sy

    fast_predictions, _ = model.roi_heads(
        pyramid, [external_boxes], transformed.image_sizes
    )
    fast_predictions = model.transform.postprocess(
        fast_predictions, transformed.image_sizes, [(H, W)]
    )
print('Fast R-CNN-style detections:',
      len(fast_predictions[0]['boxes']))
```

Cell 4.3 | Function to display predicted boxes

```
def show_detections(ax, prediction, title, threshold=0.5):
    ax.imshow(image_rgb)
    boxes = prediction['boxes'].detach().cpu().numpy()
    scores = prediction['scores'].detach().cpu().numpy()
    labels = prediction['labels'].detach().cpu().numpy()
    for box, score, label in zip(boxes, scores, labels):
        if score < threshold:
            continue
        x1, y1, x2, y2 = box
        ax.add_patch(Rectangle(
            (x1, y1), x2 - x1, y2 - y1,
            fill=False, edgecolor='#16a34a', linewidth=2
        ))
        ax.text(x1, max(0, y1 - 5),
                f'{categories[int(label)]} {score:.2f}',
                fontsize=8, color='white',
                bbox=dict(facecolor='#14532d', pad=2))
    ax.set_title(title)
    ax.axis('off')

fig, ax = plt.subplots(figsize=(9, 6))
show_detections(ax, fast_predictions[0],
                'External proposals + pretrained RoI heads')
plt.show()
```

Technical note. This is a trained Fast R-CNN-style detection stage, NOT a separate historical Fast R-CNN model. The RoI heads come from a pretrained Faster R-CNN and use RoIAlign internally; the earlier cell demonstrated the original Fast R-CNN RoI Pooling operation.

Checkpoint questions

1. Which stage still depends on Selective Search?
2. Why does reusing trained RoI heads not constitute training a new Fast R-CNN?
3. Why might some objects be missed with external proposals?

Task 5 — Faster R-CNN: learned Region Proposal Network

Replace Selective Search with an RPN while retaining the same backbone and trained RoI heads.

Cell 5.1 | Inspect anchors and generate RPN proposals

```
with torch.inference_mode():
    anchors = model.rpn.anchor_generator(
        transformed, list(pyramid.values())
    )[0]
    rpn_proposals, _ = model.rpn(transformed, pyramid)

print('Anchors in image:', anchors.shape[0])
print('RPN proposals after filtering:', rpn_proposals[0].shape[0])
print('Example anchor [x1, y1, x2, y2]:')
print(anchors[0].detach().cpu().numpy())
```

Cell 5.2 | Draw the RPN candidate boxes

```
shown_boxes = rpn_proposals[0][:35].detach().cpu().numpy().copy()
shown_boxes[:, [0, 2]] /= sx
shown_boxes[:, [1, 3]] /= sy
fig, ax = plt.subplots(figsize=(9, 6))
ax.imshow(image_rgb)
for x1, y1, x2, y2 in shown_boxes:
    ax.add_patch(Rectangle(
        (x1, y1), x2 - x1, y2 - y1,
        fill=False, edgecolor='#dc2626', linewidth=0.9
    ))
ax.set_title('Proposals learned by the RPN')
ax.axis('off')
plt.show()
```

Cell 5.3 | Use RPN proposals with the trained RoI heads

```
with torch.inference_mode():
    faster_predictions, _ = model.roi_heads(
        pyramid, rpn_proposals, transformed.image_sizes
    )
    faster_predictions = model.transform.postprocess(
        faster_predictions, transformed.image_sizes, [(H, W)]
    )

fig, ax = plt.subplots(figsize=(9, 6))
show_detections(ax, faster_predictions[0],
                'RPN proposals + pretrained RoI heads')
plt.show()
```

Checkpoint questions

1. What is the purpose of anchors and objectness scores in the RPN?
2. Which region-proposal step becomes learnable in Faster R-CNN?
3. Why are the RPN boxes not the same as final detection boxes?

Task 6 — End-to-end Faster R-CNN and confidence thresholds

Run the full model and compare its predicted detections at different score thresholds.

Cell 6.1 | Run the complete Faster R-CNN model

```
with torch.inference_mode():
    if device.type == 'cuda':
        torch.cuda.synchronize()
    start = time.perf_counter()
    final_prediction = model([image_tensor])[0]
    if device.type == 'cuda':
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - start

print(f'Full-model inference time: {elapsed:.3f} s')
print('Output keys:', list(final_prediction))
print('Detected objects:', len(final_prediction['boxes']))
```

Cell 6.2 | Visualize how the confidence threshold changes output

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, threshold in zip(axes, (0.4, 0.6, 0.8)):
    show_detections(ax, final_prediction,
                    f'Score >= {threshold}', threshold)
plt.tight_layout()
plt.show()

for threshold in (0.4, 0.6, 0.8):
    count = (final_prediction['scores'] >= threshold).sum()
    print(f'Threshold {threshold}: {int(count)} detections')
```

Checkpoint questions

1. How do confidence thresholds change the number of displayed detections?
2. Does a high confidence score guarantee a correctly localized object?
3. Why can inference time differ between CPU and GPU?

Task 7 — Final comparison and checkpoint

Summarize the algorithmic change from Fast R-CNN to Faster R-CNN.

Cell 7.1 | Compare the two proposal sources

```
comparison = {
    'Selective Search proposals': len(proposals),
    'Learned RPN proposals': int(rpn_proposals[0].shape[0]),
    'Fast R-CNN-style detections': len(fast_predictions[0]['boxes']),
    'Faster R-CNN detections': len(faster_predictions[0]['boxes'])
}
for description, value in comparison.items():
    print(f'{description:33s}: {value}')
```

Technical note. Detection counts alone cannot establish which model is more accurate. Proper comparisons require ground-truth annotations and object detection metrics such as AP at specified IoU thresholds.

Checkpoint questions

1. Identify the components shared between Fast R-CNN and Faster R-CNN.
2. Which proposal method is external and which one is learned?
3. Explain why the original R-CNN, Fast R-CNN, and Faster R-CNN are all two-stage methods.
4. Suggest an experiment using annotated images to compare proposal recall.
5. Describe one limitation of this lab compared with training Fast and Faster R-CNN from scratch.

References

R. Girshick, Fast R-CNN, ICCV 2015. https://arxiv.org/abs/1504.08083
S. Ren et al., Faster R-CNN, NeurIPS 2015. https://arxiv.org/abs/1506.01497
TorchVision: https://docs.pytorch.org/vision/main/models.html#object-detection